Преобразование и объединение таблиц

Материал продолжает работу со знакомыми ипотечными записями HMDA. Новые понятия разбираются на небольших таблицах и сохранённых ответах. Демонстрационный ноутбук содержит полный код; здесь собраны объяснения и короткие примеры.

Слайд 1. Преобразование и объединение таблиц

В наших ипотечных записях организация обозначена кодом lei. Для поиска это удобно, но в отчёте хочется видеть название. Названия лежат в отдельном справочнике. Сегодня добавим их к записям, а затем соберём таблицу с количеством решений и долей одобрений по организациям.

Сначала разберём два способа показать одни и те же счётчики: исходы можно записать в строках или разнести по столбцам. Затем найдём правило, по которому две таблицы соединяются. На маленьком примере вручную посмотрим, сколько строк должно получиться. После этого проверим результат в pandas. Итогом будет отчёт, в котором можно понять происхождение каждого числа.

Слайд 2. Таблицы и смысл строки

В decisions одна строка соответствует опубликованной ипотечной записи. Её технический номер — source_row; исход хранится в action_taken. В organizations строка описывает организацию в отчётном году. Поле organization_name содержит её опубликованное название, а lei и activity_year позволяют найти нужную запись справочника.

Теперь откроем outcome_counts. Это уже сводка: одна строка описывает сочетание организации, года и исхода. Поле n показывает, сколько исходных записей попало в это сочетание. Например, n=5 означало бы пять решений, собранных в одну строку сводки. Число строк такой таблицы и количество ипотечных записей — разные величины. Проверим размер и сумму n. Получим 500 строк сводки и 6830 исходных записей.

Таблица	Одна строка	Строк
decisions	Ипотечная запись	6830
organizations	Организация в году	470
outcome_counts	Организация, год, исход	500

Слайд 3. Длинная и широкая сводка

Рассмотрим небольшой условный пример: у организации за год пять выдач, одну одобренную заявку, по которой заявитель не принял предложение, и два отказа. Можно записать три строки: код исхода и его количество. Это длинная форма. Можно записать одну строку с тремя столбцами количеств. Это широкая форма. Записей по-прежнему восемь, меняется их представление в отчёте.

Длинная форма удобна для группировки и сравнения исходов. Широкую проще читать, когда нужен один ряд на организацию. В обоих случаях заранее называем, что означает строка и где хранится количество. Иногда вы встретите выражение tidy data: каждому признаку отведён столбец, а каждой наблюдаемой единице —

строка. Какую единицу выбрать, зависит от задачи. Сейчас проследим преобразование на реальной сводке.

Форма	Представление условного примера
Длинная	Три строки: 1 → 5; 2 → 1; 3 → 2
Широкая	Выдач 5; одобрений без принятия 1; отказов 2

Слайд 4. Счётчики по организации и исходу

Соберём длинную сводку из decisions самостоятельно. В keys_long перечислим lei, activity_year и action_taken. groupby объединит записи, у которых совпадают все три значения. size посчитает количество строк в каждой группе.

Результат пока Series: сочетания трёх полей находятся в индексе, а значения — счётчики. rename("n") даёт счётчику имя. reset_index возвращает поля группировки в обычные столбцы, и мы получаем DataFrame. Покажем первые строки и найдём в них каждое поле. Затем проверим два свойства: сочетания трёх полей не повторяются, сумма n равна числу записей decisions. Эти проверки связывают сводку с её исходными данными.

```
keys_long = ["lei", "activity_year", "action_taken"]
long = decisions.groupby(keys_long).size().rename("n")
long = long.reset_index()
```

Слайд 5. Исходы в отдельных столбцах

Откроем заранее сохранённую широкую сводку outcome_wide.csv. В каждой строке теперь организация и год. Три числовых столбца содержат количества: issued для кода 1, approved_not_accepted для кода 2, denied для кода 3. Имя approved_not_accepted длинное, зато показывает, какой именно исход имеется в виду.

Организаций с решениями в нашем отборе 314, поэтому в этой таблице 314 строк. В длинной было 500 наблюдаемых сочетаний. Такое различие ожидаемо: одна организация могла иметь несколько исходов. На экране выберем одну строку и найдём её количества в длинной таблице. Дальше получим такую форму сами, чтобы видеть, как значения распределяются по ячейкам.

```
wide_input = pd.read_csv(
    LOCAL / "outcome_wide.csv", dtype={"lei": "string"}
)
wide_input.head(4)
```

Слайд 6. Разворот через pivot

У pivot три главных аргумента. В index указываем lei и activity_year: их сочетание задаёт строку будущей таблицы. В columns передаём action_taken: его значения 1, 2 и 3 станут названиями столбцов. В values указываем n: именно этот счётчик попадёт на пересечение строки и столбца.

pivot ничего не складывает. Он берёт уже рассчитанные значения и размещает их. Поэтому для каждой будущей ячейки должно быть не больше одного значения n. В нашей длинной сводке это обеспечено группировкой и проверкой ключа. После запуска увидим составной индекс из двух полей и три столбца исходов. Некоторые ячейки пусты: соответствующий исход в этой группе не встретился. Значение такой пустоты разберём отдельно.

```
wide = long.pivot(
    index=["lei", "activity_year"],
    columns="action_taken", values="n"
)
```

Слайд 7. Когда для ячейки есть два значения

Создадим учебную копию long и повторно добавим её первую строку. pd.concat соединяет строки таблиц; ignore_index=True создаёт для результата новый последовательный индекс. Теперь одно сочетание организации, года и исхода встречается дважды. Какое из двух значений должен поместить pivot в одну ячейку? Он не выбирает за нас и выдаёт ошибку.

pivot_table позволяет задать правило объединения таких значений через aggfunc. Например, "sum" складывает их. Для отдельных реальных наблюдений сумма могла бы быть нужна. В нашей копии добавлена повторная доставка уже посчитанного счётчика, поэтому сумма завысит результат. Код выполнится, но сумма станет больше 6830. Сначала выясняем происхождение повторов, затем выбираем, требуется ли их складывать.

```
wrong_wide = replayed_long.pivot_table(
    index=["lei", "activity_year"], columns="action_taken",
    values="n", aggfunc="sum", fill_value=0
)
```

Слайд 8. Обратный разворот через melt

Теперь сделаем обратный переход от широкой таблицы к длинной. Метод melt вызывается у wide_input. В id_vars перечислим lei и activity_year: эти поля нужно сохранить при каждом счётчике. В value_vars укажем три столбца исходов. var_name="outcome" задаёт имя столбца для прежних названий, value_name="n" — имя столбца с количествами.

В результате каждая строка широкой сводки даст три строки длинной: по одной для каждого исхода. Получится 314 умножить на 3, то есть 942 строки. Некоторые из них содержат нулевой счётчик. Положительных сочетаний 500, как в первоначальной длинной таблице. Исходных ипотечных записей от разворота больше не стало: сумму n проверим отдельно.

```
restored = wide_input.melt(
    id_vars=["lei", "activity_year"],
    value_vars=outcome_columns,
    var_name="outcome", value_name="n"
)
```

Слайд 9. Ноль отсутствующего исхода

Вернёмся к пустым клеткам после pivot. Мы группировали все записи сохранённого отбора. Если для организации нет строки с кодом 2, это означает: в этом наборе записей с таким исходом не было. Его количество равно нулю. Поэтому здесь можно применить fillna(0).

Параметр reindex(columns=[1, 2, 3]) явно задаёт нужные столбцы и их порядок. После заполнения astype("int64") возвращает целочисленные счётчики. rename меняет коды столбцов на понятные названия, reset_index возвращает организацию и год из индекса в обычные поля.

Сравните это с income на прошлом занятии. Пустой income означает неизвестный доход конкретной записи. Отсутствующий исход в полной сводке означает

известное количество ноль. Заполнение зависит от смысла клетки и полноты набора.

```
wide_complete = wide.reindex(columns=[1, 2, 3])
wide_complete = wide_complete.fillna(0).astype("int64")
```

Слайд 10. Проверка обратного преобразования

В restored названия исходов текстовые, а в первоначальном long использовались коды. Создадим словарь code_map и применим map к столбцу outcome. map заменит каждое известное название его кодом. Затем оставим строки с n больше нуля: именно такие сочетания присутствовали в исходной длинной сводке.

Для сравнения выберем одинаковые четыре поля: три поля ключа и n. sort_values упорядочит строки по ключу. reset_index(drop=True) уберёт старые метки индекса, которые зависели от порядка разворота. assert_frame_equal сравнит значения, типы и структуру двух таблиц. Если проверка проходит, восстановленные наблюдаемые сочетания совпадают с первоначальными. Одной суммы было бы мало: одинаковый итог можно получить и при неверном распределении счётчиков по организациям.

```
expected = long.sort_values(keys_long).reset_index(drop=True)
actual = roundtrip.sort_values(keys_long).reset_index(drop=True)
assert_frame_equal(actual, expected)
```

Слайд 11. Ключ справочника

Перейдём к названиям организаций. У organizations есть organization_name — опубликованное название, lei — код и activity_year — отчётный год. Поле source_count пришло из другого, более широкого отбора источника. Для долей по нашим 6830 записям мы будем считать количества самостоятельно.

Чтобы найти строку справочника, используем пару lei и activity_year. Такое сочетание полей называют составным ключом. Проверим, что справа оно известно и встречается один раз. duplicated(org_keys) ищет повторы сочетаний, any сообщает, есть ли хотя бы один. isna().any().any() проверяет пропуски сначала по столбцам, затем во всей выбранной части таблицы. Имя организации для поиска не используем: написание может меняться, а код и год задают нужное соответствие.

Поле	Назначение
lei	Код организации
activity_year	Отчётный год
organization_name	Название для отчёта
source_count	Счётчик другого отбора источника

Слайд 12. Сколько совпадений ожидаем

Представьте две ипотечные записи одной организации за 2023 год. Справа в справочнике для этой организации и года есть одна строка. Каждая запись слева найдёт её и получит название. В результате останутся две строки.

Такую связь называют «многие к одному», many-to-one: много ипотечных записей относятся к одной строке справочника. Если ключи уникальны в обеих таблицах, связь называется «один к одному». Если они повторяются с обеих сторон, для одного ключа образуются все пары совпадений. Например, две левые строки и две правые дадут четыре пары. Поэтому ожидаемое число совпадений нужно назвать до соединения. В нашем задании справа требуется одна строка на организацию и год.

Связь	Ключ слева	Ключ справа
Один к одному	Уникален	Уникален
Многие к одному	Может повторяться	Уникален
Многие ко многим	Повторяется	Повторяется

Слайд 13. Соединение на двух записях

Возьмём из настоящего справочника LOANDEPOT.COM, LLC. Выберем две её записи из decisions и покажем рядом одну строку organizations. Полные коды lei видны в ноутбуке. Сначала найдите одинаковый код и год глазами. Название из правой строки нужно дописать к каждой из двух левых.

Эту операцию называют соединением таблиц, JOIN. В pandas её выполняет merge. Слева вызываем метод у left_example, справа передаём right_example. on=org_keys задаёт поля для сопоставления, how="left" сохраняет левые записи, validate="many_to_one" проверяет уникальность правого ключа. Выполним вызов и сравним результат с ручным ожиданием: два прежних source_row, два прежних исхода и добавленное название.

source_row	action_taken	Добавленное имя
7593	1	LOANDEPOT.COM, LLC
7596	1	LOANDEPOT.COM, LLC

```
hand_join = left_example.merge(
    right_example, on=org_keys, how="left",
    validate="many_to_one"
)
```

Слайд 14. left и inner при неполном справочнике

Сделаем учебную копию справочника без первой организации. В нашем отборе ей соответствуют 25 записей. Сначала выполним left-соединение. Эти записи сохранятся, но добавленное название у них останется неизвестным.

Теперь сменим только how на "inner". Получим записи, для которых нашлось совпадение справа. Двадцать пять строк исчезнут, и результат сократится до 6805. Оба вызова могут пройти validate="many_to_one": справа ключ по-прежнему уникален. Проверка уникальности отвечает на вопрос о числе возможных соответствий, а проверка покрытия — о том, всем ли записям нашлось название. Какой вариант нужен, определяется вопросом. Для добавления имён без потери ипотечных записей выбираем left.

```
left_partial = decisions.merge(
    partial_orgs, on=org_keys, how="left",
    validate="many_to_one", indicator=True
)
```

Слайд 15. Полное соединение и проверка строк

Выполним соединение полного decisions с полным organizations. В on передадим два поля, lei и activity_year. how="left" сохраняет левую таблицу. validate="many_to_one" останавливает выполнение, если справа один ключ встречается несколько раз. indicator=True добавляет служебный столбец _merge, по которому увидим, найдено ли соответствие.

merge возвращает новый DataFrame. Проверим его размер, уникальность source_row и совпадение множества исходных номеров. Затем покажем source_row, organization_name и action_taken рядом. Каждой ипотечной записи должно соответствовать одно название организации за нужный год. Проверка размера особенно полезна здесь: неправильный справочник способен увеличить число строк, хотя мы просили только добавить название.

```
enriched = decisions.merge(
    organizations, on=org_keys, how="left",
    validate="many_to_one", indicator=True
)
```

Слайд 16. Повтор справочника умножает строки

Добавим к справочнику повторную строку первой организации. Вызов без validate выполнится: каждая из 25 её ипотечных записей найдёт теперь два совпадения. Получится 6855 строк вместо 6830. Остальные организации на это увеличение не повлияли.

Посчитаем повторные source_row в результате: они помогают обнаружить, какие исходные записи размножились. Затем повторим вызов с validate="many_to_one". pandas выдаст MergeError, потому что правая таблица нарушает ожидаемую уникальность. В примере мы перехватываем именно эту ожидаемую ошибку через try/except, чтобы продолжить лекцию. В своей работе сначала исправляем состав справочника, затем выполняем соединение заново.

Справочник	Строк результата	Повторные source_row
Полный исходный	6830	0
Учебная копия с повтором	6855	25

Слайд 17. Каким записям не нашлось имени

Посмотрим на _merge, добавленный indicator=True. Значение both означает, что для левой записи найден правый ключ. Значение left_only означает, что левая запись осталась без соответствия. Для полного справочника все 6830 записей имеют both.

В учебной неполной копии есть 25 left_only. Выберем их маской и покажем source_row, lei и activity_year. Такая отдельная таблица позволяет проверить, какого имени не хватает и какие записи затронуты. При left JOIN неизвестное название не означает неизвестный исход: действие по заявке осталось в исходном поле action_taken. Поэтому показатель по решениям можно проверить отдельно, явно сохраняя записи без справочного имени.

```
enriched["_merge"].value_counts(dropna=False)
left_partial.loc[left_partial["_merge"].eq("left_only")]
```

Слайд 18. Пропуски в ключе

Рассмотрим отдельные технические копии наших маленьких таблиц. В обеих заменим lei пропуском и попробуем inner-соединение. pandas может сопоставить неизвестные значения ключей; в примере получится две строки. Но отсутствие кода организации не даёт нам основания утверждать, что это одна организация.

Поэтому перед рабочим соединением проверяем известность обоих полей ключа, а строки с неизвестным ключом сохраняем для отдельного разбора. Ранее мы уже различали неизвестный доход и ноль. Здесь похожая задача чтения смысла:

одинаковое обозначение отсутствия значения не доказывает одинаковую сущность. После демонстрации вернёмся к исходным таблицам, в которых ключи известны.

```
unknown_left["lei"] = pd.NA
unknown_right["lei"] = pd.NA
null_pairs = unknown_left.merge(
    unknown_right, on=org_keys, how="inner"
)
```

Слайд 19. Какие поля брать из справочника

Для добавления имени не обязательно переносить все столбцы organizations. Создадим name_lookup: возьмём ключи и organization_name. Соединим его с decisions и прочитаем список полученных столбцов. source_count сюда не попадёт, а исходные ипотечные поля останутся на месте.

Если у обеих таблиц есть одноимённые поля помимо ключа, merge по умолчанию добавляет суффиксы _x и _y. Аргумент suffixes позволяет назвать стороны понятнее, например _left и _right. Суффикс показывает происхождение поля, но не выбирает за нас правильное значение. В сегодняшнем результате лишних одинаковых полей можно избежать заранее: выбрать из справочника именно то, что нужно для отчёта.

```
name_lookup = organizations[
    org_keys + ["organization_name"]
].copy()
```

Слайд 20. Профиль организаций с именами

Соберём итоговый профиль. Сначала в копии decisions создаём логический столбец is_approved: True для исходов 1 и 2. groupby по организации и году задаёт группы. В agg поле n_decisions получает число строк, а n_approved — сумму True. Затем делим второй счётчик на первый внутри каждой строки профиля.

После группировки каждая организация и год встречаются один раз. Справа в name_lookup этот ключ тоже уникален, поэтому для добавления имени используем validate="one_to_one". Получим 314 строк. Проверим суммы: 6830 решений и 6139 одобрений. Доля каждой организации имеет собственный знаменатель. Чтобы восстановить общую долю, складываем количества одобрений и решений и только потом делим; простое среднее процентов дало бы каждой организации одинаковый вес.

```
profile["approval_rate"] = (
    profile["n_approved"] / profile["n_decisions"]
)
```

Слайд 21. Разбор неверного соединения

Представьте, что коллега прислал таблицу из 6855 строк и объяснил увеличение тем, что к данным добавили названия. Для нашей задачи это повод проверить соединение. Добавление одного справочного имени к каждой записи должно сохранить число ипотечных строк.

Предложите порядок проверки: посчитать повторные source_row, посмотреть затронутые lei и год, затем проверить эти ключи справа. После этого выяснить, перед нами одинаковая повторная доставка или разные версии названия. В нашем учебном примере причина уже известна: первая строка справочника была добавлена повторно. Исправим её до merge, включим validate и снова проверим состав, значения исходных полей и покрытие. Это законченный разбор ошибки, который можно воспроизвести на практике.

Слайд 22. Практика преобразования и JOIN

Сегодня мы сделали две разные операции. pivot и melt меняли представление счётчиков. merge добавлял сведения из другой таблицы по общим полям. Для разворота проверяли сочетания и количества, для соединения — ожидаемое число совпадений, сохранность записей и найденные имена.

На практике P04 вы повторите эти действия по шагам. В ноутбуке будут готовые примеры и пустые ячейки A–E для ваших решений. Условия и словарь находятся в методичке, письменные объяснения — в отдельном отчёте. После практики домашняя работа запускается самостоятельно: вы построите профиль отказов и выполните обратное преобразование формы. Дополнительная функция безопасного соединения остаётся необязательной.

Источники и повторение

Словарь и происхождение данных (../../../practices/P04/data/card.md). Общий снимок HMDA (../../../data/card.md).

Документация pandas 2.2: pivot (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.DataFrame.pivot.html), pivot_table (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.DataFrame.pivot_table.html), melt (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.DataFrame.melt.html), merge (https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.DataFrame.merge.html).